

# SOC con Respuesta Automatizada (SOAR)

*Centro de Operaciones de Seguridad open-source con detección multicapa y  
respuesta orquestada sobre Proxmox VE*

---

## Memoria Técnica

**CVL**

Proyecto de Síntesis

Junio de 2026



## 1. Introducción

---

La creciente sofisticación de las amenazas obliga a las organizaciones a dotarse de capacidades de detección y respuesta que vayan más allá de la simple recolección de registros. Un Centro de Operaciones de Seguridad (SOC) moderno no solo debe observar lo que ocurre en la infraestructura, sino correlacionar esa información, enriquecerla con inteligencia de amenazas y reaccionar de forma rápida y consistente.

Este proyecto demuestra la construcción de un SOC completo y funcional empleando exclusivamente herramientas de código abierto, desplegado sobre el hipervisor Proxmox VE. El sistema integra detección en tres capas independientes (kernel, sistema y red), correlación centralizada en un SIEM/XDR y un motor de respuesta automatizada (SOAR) que encadena el enriquecimiento, el análisis asistido por inteligencia artificial local, la apertura de casos y la notificación al analista.

El resultado es una plataforma que cubre el ciclo de vida completo de un incidente, desde la detección hasta la respuesta, en menos de un minuto, y que sirve tanto como laboratorio de aprendizaje como referencia de diseño reproducible.

## 2. Objetivos

---

El proyecto persigue los siguientes objetivos técnicos:

- Implementar una arquitectura SOC completa y reproducible sobre Proxmox VE.
- Integrar tres capas de telemetría complementarias: kernel (eBPF), sistema (Falco) y red (NIDS).
- Establecer un pipeline de detección, correlación y respuesta automatizada de extremo a extremo.
- Utilizar un modelo de lenguaje ejecutado localmente para el análisis semántico de las alertas, sin enviar datos a terceros.
- Demostrar capacidad de bloqueo activo a nivel de kernel mediante políticas de eBPF.
- Documentar el diseño de forma defendible, distinguiendo lo validado de la arquitectura de referencia.

## 3. Arquitectura de red

---

La red se diseña segmentada en VLANs sobre Proxmox VE, con un firewall pfSense que actúa como router y punto de control entre zonas. La segmentación aísla la zona de seguridad (SOC) de los servicios expuestos (DMZ) y del tráfico no confiable del atacante, aplicando una política de mínimo privilegio entre segmentos.

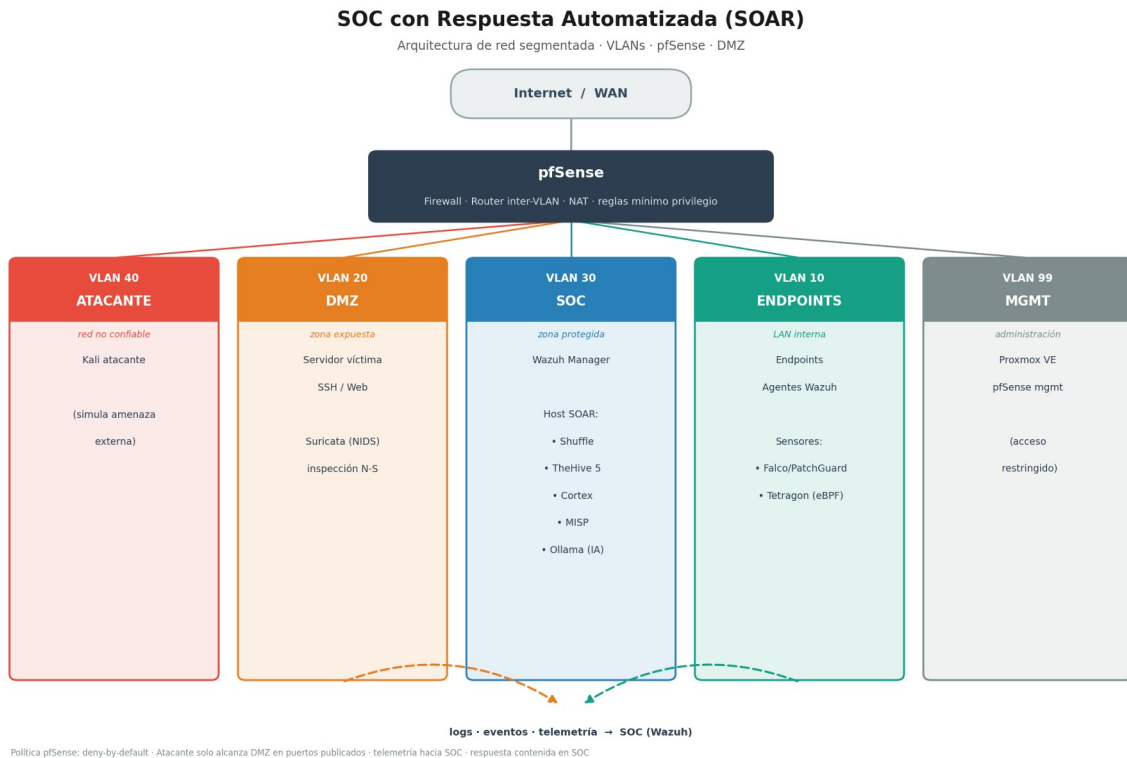


Figura 1. Arquitectura de red segmentada: VLANs, pfSense y DMZ.

### 3.1. Segmentos (VLANs)

La infraestructura se divide en cinco segmentos, cada uno con un nivel de confianza y un propósito definidos:

| VLAN | Zona       | Confianza                | Contenido                                   |
|------|------------|--------------------------|---------------------------------------------|
| 40   | Atacante   | Ninguna (no confiable)   | Kali atacante; simula amenaza externa       |
| 20   | DMZ        | Baja (expuesta)          | Servidor víctima (SSH/Web), Suricata (NIDS) |
| 30   | SOC        | Alta (protegida)         | Wazuh Manager, host del stack SOAR          |
| 10   | Endpoints  | Media (LAN interna)      | Endpoints con agentes Wazuh y sensores      |
| 99   | Management | Crítica (administración) | Proxmox VE, gestión de pfSense              |

En Proxmox, el bridge virtual se configura en modo VLAN-aware y cada máquina se etiqueta con su identificador de VLAN. pfSense recibe todas las VLANs y enruta y filtra el tráfico entre ellas.

### 3.2. Política de firewall

La política base es de denegación por defecto: solo se permite explícitamente el tráfico necesario. La siguiente matriz resume las reglas inter-zona:

| Origen → Destino                  | Política                                          |
|-----------------------------------|---------------------------------------------------|
| Atacante → DMZ                    | Permitido solo en puertos publicados (22, 80/443) |
| Atacante → SOC / Endpoints / Mgmt | Denegado                                          |
| DMZ → SOC                         | Solo envío de logs y eventos (agente Wazuh)       |
| DMZ → Endpoints / Atacante        | Denegado                                          |

| Origen → Destino        | Política                                          |
|-------------------------|---------------------------------------------------|
| Endpoints → Internet    | Permitido (NAT)                                   |
| Endpoints → SOC         | Solo telemetría (agente Wazuh)                    |
| SOC → Internet          | Limitado (feeds de inteligencia, actualizaciones) |
| Cualquiera → Management | Denegado (salvo jump host de administración)      |

### 3.3. Flujos de tráfico legítimos

El tráfico de seguridad nunca atraviesa la DMZ ni la red del atacante. La telemetría viaja en sentido Endpoints/DMZ hacia el SOC, y la respuesta queda contenida dentro de la zona SOC:

1. Detección: los sensores en DMZ y Endpoints envían sus eventos al gestor de Wazuh.
2. Correlación: Wazuh aplica las reglas y, al cruzar el umbral, dispara el workflow de Shuffle.
3. Respuesta: Shuffle orquesta el enriquecimiento, el análisis y la gestión del caso dentro del SOC.
4. Notificación: se envía una tarjeta resumen al canal del analista.
5. Administración: las consolas solo son accesibles desde la zona de gestión o un jump host.

## 4. Stack tecnológico

El sistema se compone íntegramente de software de código abierto. La siguiente tabla resume cada componente y su función dentro del SOC:

| Tecnología         | Función                                               |
|--------------------|-------------------------------------------------------|
| Proxmox VE         | Hipervisor; virtualización del laboratorio            |
| pfSense            | Firewall, router inter-VLAN y NAT                     |
| Wazuh              | SIEM/XDR/EDR; correlación y gestión de alertas        |
| Tetragon (eBPF)    | Observabilidad y bloqueo a nivel de kernel            |
| Falco              | Detección de anomalías por syscalls (capa de sistema) |
| Suricata           | IDS/NIDS; análisis de tráfico de red                  |
| Shuffle            | SOAR; orquestación de la respuesta automatizada       |
| TheHive 5          | Gestión de casos e incidentes                         |
| Cortex             | Motor de analizadores y enriquecimiento               |
| MISP               | Plataforma de inteligencia de amenazas (IOC)          |
| Ollama             | Modelo de lenguaje local para análisis semántico      |
| Docker + Portainer | Containerización y gestión de microservicios          |

## 5. Capas de detección

El SOC observa la actividad en tres niveles independientes y complementarios. La redundancia entre capas aumenta la probabilidad de detección y permite contrastar señales de distinto origen:

| Capa   | Sensor          | Función                                                          |
|--------|-----------------|------------------------------------------------------------------|
| Kernel | Tetragon (eBPF) | Monitoriza el kernel y permite bloqueo activo: una TracingPolicy |

| Capa    | Sensor   | Función                                                                                                                                   |
|---------|----------|-------------------------------------------------------------------------------------------------------------------------------------------|
|         |          | termina con SIGKILL cualquier proceso que lea ficheros sensibles como las credenciales del sistema.                                       |
| Sistema | Falco    | Detecta comportamientos anómalos por llamadas al sistema: shells interactivas, procesos sospechosos o modificación de tareas programadas. |
| Red     | Suricata | Inspecciona el tráfico de red por firmas conocidas: escaneos, exploits y descargas.                                                       |

Todas las capas envían sus eventos al gestor de Wazuh, que actúa como punto central de correlación. Conviene destacar que, en una red conmutada sin replicación de puertos, el sensor de red solo observa el tráfico de su propio segmento; aun así, acciones como la descarga de un payload se detectan igualmente en la capa de sistema por la ejecución del proceso correspondiente.

## 6. Pipeline de respuesta automatizada (SOAR)

Cuando una alerta supera el umbral configurado, el gestor de Wazuh dispara, mediante un webhook, el workflow del orquestador SOAR. El flujo encadena los siguientes pasos:

6. Recepción de la alerta y extracción de los indicadores de compromiso (IOC).
7. Enriquecimiento de los IOC contra la plataforma de inteligencia de amenazas.
8. Análisis semántico de la alerta mediante un modelo de lenguaje local.
9. Normalización del resultado del análisis.
10. Creación de una alerta y su promoción a caso en la plataforma de gestión de incidentes.
11. Envío de una notificación con el resumen, la severidad, el activo afectado, el IOC y el enlace al caso.

*[ Espacio para captura: workflow del orquestador SOAR ]*

### 6.1. Integración con el SIEM

La integración se realiza mediante un script personalizado en el gestor de Wazuh que filtra las alertas por los grupos de los sensores del SOC y reenvía únicamente las relevantes al webhook del orquestador, preservando la estructura del evento para que el workflow pueda resolver sus campos. El umbral de disparo debe calibrarse con la severidad real de los eventos: un umbral demasiado alto impide que los ataques activen el workflow, mientras que uno demasiado bajo satura la cola de análisis.

*[ Espacio para captura: alerta en el panel del SIEM ]*

### 6.2. Análisis con inteligencia artificial local

El análisis semántico se ejecuta con un modelo de lenguaje ligero alojado localmente, lo que garantiza que ningún dato sensible del SOC abandone el laboratorio. El modelo resume la alerta y valora su gravedad en lenguaje natural, aportando contexto al analista sin depender de servicios externos. Para mantener tiempos de respuesta razonables en CPU se emplean instrucciones cortas y se limita la longitud de la generación.

## 7. Reglas de detección

Las reglas personalizadas se organizan en una serie propia y se agrupan por sensor. Cada regla asocia una detección a un nivel de severidad y, cuando procede, a una técnica de MITRE ATT&CK. Entre ellas se incluye la elevación de severidad de la lectura de credenciales del sistema, que es la acción crítica que desencadena la respuesta automatizada, así como una regla de supresión que neutraliza un falso positivo recurrente generado por el propio sensor de red, evitando que inunde el SIEM con alertas idénticas.

## 8. Cadena de ataque y validación

Para validar la detección y la respuesta se emula una cadena de ataque de seis fases contra el servidor de la DMZ. Cada fase está mapeada a la técnica de MITRE ATT&CK correspondiente y al sensor que la detecta:

| Fase               | Acción                              | Sensor             | MITRE ATT&CK          |
|--------------------|-------------------------------------|--------------------|-----------------------|
| 1 — Acceso inicial | Fuerza bruta SSH                    | Wazuh              | T1110                 |
| 2 — Ejecución      | Shell interactiva                   | Falco              | T1059.004             |
| 3 — Descubrimiento | Reconocimiento del sistema          | Tetragon / Falco   | T1033 / T1082 / T1087 |
| 4 — Credenciales   | Lectura de credenciales del sistema | Tetragon / Falco   | T1003.008             |
| 5 — Transferencia  | Descarga de payload                 | Falco (+ Suricata) | T1105                 |
| 6 — Persistencia   | Tarea programada maliciosa          | Tetragon / Falco   | T1053.003             |

La cuarta fase es la acción crítica: la lectura de las credenciales del sistema cruza el umbral y desencadena el pipeline completo. La emulación se automatiza con pausas entre fases para no saturar la cola de análisis. Si la política de bloqueo a nivel de kernel está activa, el proceso de esa fase se termina de inmediato.

*[ Espacio para captura: caso creado en la plataforma de gestión ]*

*[ Espacio para captura: tarjeta de notificación con el análisis ]*

## 9. Despliegue

Los servicios se despliegan como pilas de contenedores gestionadas con Portainer, organizadas en una pila de telemetría (sensores) y otra de respuesta (orquestador, gestión de casos, inteligencia y modelo de lenguaje). El gestor del SIEM se ejecuta en una máquina independiente.

Durante la integración se consolidaron varios patrones de configuración: la base de datos de búsqueda requiere ajustes de memoria del anfitrión y de propiedad de los volúmenes; el orquestador debe referirse a sus servicios por el nombre interno de red en lugar de por dirección, y emplear las imágenes oficiales actuales; y la plataforma de inteligencia regenera parte de su configuración en cada arranque, por lo que los ajustes deben aplicarse mediante variables de entorno. Asimismo, las rutas de montaje deben ser absolutas para una resolución consistente.

## 10. Resultados

---

El pipeline de extremo a extremo quedó plenamente operativo: una alerta del SIEM desencadena el workflow del orquestador, que crea un caso en la plataforma de gestión y emite una notificación con el análisis generado por la inteligencia artificial, la severidad, el activo afectado, el indicador de compromiso y el enlace al caso. El conjunto se ejecuta de forma fiable y en un tiempo reducido, demostrando la viabilidad de un SOC con respuesta automatizada construido íntegramente con software libre.

## 11. Lecciones aprendidas

---

La puesta en marcha del pipeline dejó varias enseñanzas relevantes:

- La exportación fiable de datos entre nodos de automatización exige respetar estrictamente los formatos de salida y los nombres de los nodos; pequeñas inconsistencias provocan fallos silenciosos.
- La creación de alertas en la plataforma de gestión requiere campos obligatorios y referencias únicas para evitar la deduplicación silenciosa de incidentes.
- Inyectar objetos complejos en las peticiones al modelo de lenguaje rompe el formato; conviene usar texto y campos simples previamente normalizados.
- El umbral de disparo debe calibrarse con la severidad real de los sensores para equilibrar cobertura y rendimiento.
- Los falsos positivos de alto volumen deben suprimirse de forma específica para no degradar la señal del SIEM.

## 12. Conclusiones

---

El proyecto confirma que es posible construir un SOC completo, con detección multicapa y respuesta automatizada, empleando únicamente herramientas de código abierto sobre una plataforma de virtualización accesible. La combinación de observabilidad a nivel de kernel, sistema y red, junto con una orquestación que integra inteligencia de amenazas y análisis asistido por IA local, ofrece un equilibrio notable entre capacidad, coste y privacidad. El diseño es reproducible y sienta una base sólida para evolucionar hacia un entorno de producción segmentado.

## 13. Trabajo futuro

---

- Habilitar la replicación de puertos para dar al sensor de red visibilidad completa del tráfico interno.
- Incorporar respuestas activas adicionales, como el bloqueo automático de direcciones.
- Integrar frameworks de emulación de adversario para ampliar la batería de pruebas.